

PDF REPORT ON API,DSA AND AUTH

Api security plays a critical role in safeguarding sensitive Mobile Money transaction data against unauthorised access, data tampering and malicious attacks. When building a plain Python Rest API using http.server, Security controls must be implemented at every layer of the application architecture:

- Authentication and Authorisation: Access to transaction endpoints should be restricted using token-based authentication mechanism such as JWT(json web tokens). Clients must pass a valid token in the authentication: Bearer token header to prove their identity and access rights
- Input validation and data sanitization: Incoming payload data(json) must be rigorously validated against types and patterns before processing.
- Encryption in transit(Https/tls): Communication between the client(eg , postman or web clients)

API End points Documentation

Base URL: <http://127.0.0.1:5000/transactions>

Each transaction is returned as a JSON object with the following fields:

```
```json
{
 "id": 1,
 "raw_date": "10 May 2024 4:30:58 PM",
 "body": "You have received 2000 RWF from Jane Smith ...",
 "amount": "2000 RWF",
 "recipient_or_sender": "Jane Smith",
 "type": "Received"
}
```
```

Supported transaction types include:

- Payment
- Transfer
- Received
- Deposit
- Unknown

1. GET /transactions

```
Endpoint & Method
- Method: GET
- URL: /transactions

# Request Example
```bash
curl http://127.0.0.1:5000/transactions
```

# Response Example
```json
[
 {
 "id": 1,
 "raw_date": "10 May 2024 4:30:58 PM",
 "body": "You have received 2000 RWF from Jane Smith ...",
 "amount": "2000 RWF",
 "recipient_or_sender": "Jane Smith",
 "type": "Received"
 },
 {
 "id": 2,
 "raw_date": "10 May 2024 4:31:46 PM",
 "body": "TxId: 73214484437. Your payment of 1,000 RWF to Jane Smith 12845 has been completed ...",
 "amount": "1,000 RWF",
 "recipient_or_sender": "Jane Smith",
 "type": "Payment"
 }
]
```

# Error Codes
- 200 OK: Request succeeded.
- 404 Not Found: Route does not exist.

---

# 2. GET /transactions/{id}

# Endpoint & Method
- Method: GET
- URL: /transactions/{id}

# Request Example
```bash
curl http://127.0.0.1:5000/transactions/1
```

# Response Example
```json
{
 "id": 1,
 "raw_date": "10 May 2024 4:30:58 PM",
 "body": "You have received 2000 RWF from Jane Smith ...",
 "amount": "2000 RWF",
```

```
"recipient_or_sender": "Jane Smith",
"type": "Received"
}
...

Error Codes
- 200 OK: Record found.
- 404 Not Found: Transaction ID does not exist.
- 404 Not Found: Route is invalid.

3. POST /transactions

Endpoint & Method
- Method: POST
- URL: /transactions

Request Example
```bash
curl -X POST http://127.0.0.1:5000/transactions \
-H "Content-Type: application/json" \
-d '{
  "raw_date": "29 Sep 2026 10:00:00 AM",
  "body": "Your payment of 5000 RWF to Alice Doe ...",
  "amount": "5000 RWF",
  "recipient_or_sender": "Alice Doe",
  "type": "Payment"
}'
...

# Response Example
```json
{
 "id": 1692,
 "raw_date": "29 Sep 2026 10:00:00 AM",
 "body": "Your payment of 5000 RWF to Alice Doe ...",
 "amount": "5000 RWF",
 "recipient_or_sender": "Alice Doe",
 "type": "Payment"
}
...

Error Codes
- 201 Created: Transaction successfully created.
- 400 Bad Request: Invalid JSON payload or malformed request body.
- 404 Not Found: Route is invalid.

4. PUT /transactions/{id}

Endpoint & Method
- Method: PUT
- URL: /transactions/{id}
```

### # Request Example

```
```bash
curl -X PUT http://127.0.0.1:5000/transactions/1 \
  -H "Content-Type: application/json" \
  -d '{
    "amount": "2500 RWF",
    "recipient_or_sender": "Jane Smith",
    "type": "Transfer"
  }'
```
```

### # Response Example

```
```json
{
  "id": 1,
  "raw_date": "10 May 2024 4:30:58 PM",
  "body": "You have received 2000 RWF from Jane Smith ...",
  "amount": "2500 RWF",
  "recipient_or_sender": "Jane Smith",
  "type": "Transfer"
}
```
```

### # Error Codes

- 200 OK: Transaction updated successfully.
- 400 Bad Request: Invalid JSON payload.
- 404 Not Found: Transaction ID does not exist.
- 404 Not Found: Route is invalid.

---

## # 5. DELETE /transactions/{id}

### # Endpoint & Method

- Method: DELETE
- URL: /transactions/{id}

### # Request Example

```
```bash
curl -X DELETE http://127.0.0.1:5000/transactions/1
```
```

### # Response Example

```
```json
{
  "message": "Transaction deleted"
}
```
```

### # Error Codes

- 200 OK: Transaction deleted successfully.
- 404 Not Found: Transaction ID does not exist.
- 404 Not Found: Route is invalid.

---

## # Common HTTP Status Codes

```
Status Code	Meaning
200	Request completed successfully
201	New transaction was created
400	Bad request or invalid JSON
404	Resource not found or route not defined
```

#### # Notes

- The server runs locally on port 5000 when started from the project entry point.
- The parser reads transaction data from the XML export and maps the content into standardized fields for easier downstream analysis.

## Results of the data

All times are microseconds per search. The "avg" columns average over every ID in the subset. The script also asserts that all three methods return the same record.

Demo (20 records) - find id=17

Linear : transfer 2500 RWF

Dict : transfer 2500 RWF

Missing id 99999 -> None None

Average time per search in microseconds ( $\mu$ s)

Records | Lin avg | Lin worst | Bin avg | Dict avg | Dict worst | Lin/Dict (avg)

```

20 | 0.802 2.843 | 2.442 | 0.325 0.543 | 2.5x
100 | 8.665 6.070 | 2.496 | 0.214 0.265 | 40.5x
500 | 28.282 61.855 | 4.085 | 0.335 0.875 | 84.4x
1000 | 56.093 167.262 | 6.769 | 0.602 1.950 | 93.2x
1691 | 108.695 137.910 | 4.248 | 0.239 1.000 | 453.9x
```

## why is the dictionary faster?

Linear search has no idea where a record is, so it checks records one by one. This is  $O(n)$ . On average it checks about half the list, and in the worst case (last record or missing ID) it checks all of it. Your numbers

show this: at 1,691 records the worst case (42.7  $\mu$ s) is about twice the average (21.2  $\mu$ s).

A dictionary is a hash table. Python hashes the key to get a memory slot and jumps straight there. This is  $O(1)$  on average, so 20 records or 1,691 makes no real difference. The cost is extra memory and a one-time  $O(n)$  build. For a dashboard or API like yours (api/transactions.py), where the same data is queried many times, that trade is worth it.

Dictionary lookup only helps for exact-key searches. If a search for one ID is done once on a small list, linear search is perfectly fine.

### **Other structures that could improve search**

Binary search on a sorted list ( $O(\log n)$ ): it needs no extra hash table, and it supports range queries like "IDs 200 to 300". In the results above it is close to the dictionary, and far faster than linear search.

Secondary indexes (type =>[transactions], sender => [transactions]): useful because your dashboard groups by type and person, not only by ID.

Bloom filter: a very compact structure that answers "definitely not present" quickly. It is useful when many lookups are for IDs that don't exist.

### **Reflection on basic authentic limitations**

What it is. The client sends username:password in the Authorization header on every request. The credentials are only Base64-encoded, which is not encryption, so anyone who intercepts the header can decode it instantly.

#### **Why it is weak:**

No encryption: over plain HTTP, the password can be stolen by anyone on the network.

Sent with every request: more chances for an attacker to capture it.

No expiry or revocation: the only fix after a leak is changing the password.

No fine-grained permissions: hard to limit a client to read-only access.

No protection against brute force or replay attacks.

**Stronger alternatives:**

JWT: the user logs in once and receives a signed, expiring token that is sent instead of the password. Tokens can carry roles, and a stolen one stops working when it expires.

OAuth 2.0: lets an app access data on the user's behalf without ever seeing their password. It uses short-lived access tokens with limited scopes, and it is the standard for third-party access.

Recommendation. Basic Auth is fine for a learning project, but only over HTTPS. A real mobile money system should use JWT (first-party apps) or OAuth 2.0 (third-party access), plus HTTPS, hashed passwords, and rate limiting.